

OpenText™ Identity Manager

Entitlements Guide

Version : 25.4

PDF Generated on : April 21, 2026

Table of Contents

1. Entitlements Guide	1
1.1. Understanding Entitlements	2
1.1.1. How Entitlements Work	3
1.1.2. Why Use Entitlements	5
1.1.3. Drivers with Preconfigured Entitlements	6
1.1.4. Entitlements Formats	7
1.1.5. Understanding Identity Manager 4.0 and Later Entitlement Formats	8
1.2. Checklist for Implementing Entitlements	14
1.3. Enabling Entitlements on a Driver	17
1.3.1. Using Designer to Enable Entitlements	18
1.4. Creating Entitlements	19
1.4.1. Sample Entitlements for the Active Directory Driver	20
1.4.2. Creating Entitlements in OpenText Designer	21
1.5. Creating Policies to Support Entitlements	31
1.6. Editing Entitlements in Designer	34
1.7. Writing Entitlements in XML	42
1.7.1. Entitlement Document Type Definition	43
1.7.2. Examples to Help You Write Your Own Entitlements	49

1. Entitlements Guide

This guide explains how to create and use entitlements as part of your OpenText Identity Manager solution. This guide is intended for OpenText Identity Manager administrators, partners, and consultants.

1.1. Understanding Entitlements

OpenText™ Identity Manager uses entitlements as a way for you to provide users with access to resources in connected systems.

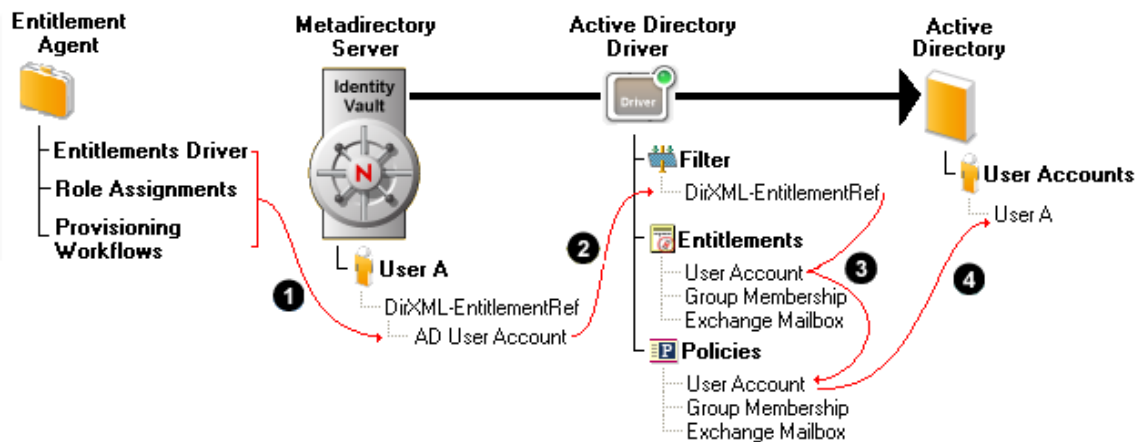
An entitlement is a permission slip. For example, to grant a new employee an Active Directory account after their addition to your Human Resource system, the employee must possess an entitlement, akin to a permission slip, for the Active Directory account. If the user does not have the permission slip, they do not receive the account.

The following sections explain how entitlements work and how they make the administration of your OpenText Identity Manager system more efficient.

- [How Entitlements Work](#)
- [Why Use Entitlements?](#)
- [Drivers with Preconfigured Entitlements](#)
- [Entitlements Formats](#)
- [Understanding Identity Manager 4.0 and Later Entitlement Formats](#)

1.1.1. How Entitlements Work

The following diagram illustrates the basic entitlement process:



Overview of Entitlements

1. An entitlement agent grants an entitlement to a user. There are three ways that entitlements are granted to a user:
 - **Role-Based Entitlements:** The Entitlements Service driver grants the entitlement based on criteria that place the user in a particular role (or group). The criteria can be based on any event that occurs in the Identity Vault. For example, adding a new employee in an HR system causes a User object to be created in the Identity Vault. Creation of the new User object is the criterion that causes the Entitlements Service driver to grant the Active Directory User Account entitlement to the user.
 - **User Application Roles Based Provisioning:** The user receives a role assignment through the User Application. The User Application's Role Service driver grants the user any entitlements associated with the new role. For example, a user is assigned an Accountant role that requires access to the Accounting group in Active Directory. The Role Service driver grants the Active Directory Group Membership entitlement to the user.
 - **User Application Workflow-Based Provisioning:** A provisioning workflow grants the entitlement to the user. For example, a new employee is added to the HR system, which causes a User object to be created in the Identity Vault. Creation of the new User object initiates a workflow that grants the Active Directory User Account entitlement to the user.
2. When an entitlement is added to or removed from a user's DirXML-EntitlementRef attribute, any entitlement-enabled drivers begin to process the

event. Only drivers that have the DirXML-EntitlementRef attribute added to their Subscriber channel filter can monitor users for entitlement changes.

3. The driver processes the entitlement event against the Subscriber channel policies. If the entitlement event is for an entitlement that applies to the driver, the policies are processed. Otherwise, no processing occurs. In the diagram above, the Grant User Account policy is processed because 1) the Active Directory User Account entitlement was added to the user's DirXML-EntitlementRef attribute and 2) the User Account entitlement is defined on the Active Directory driver. If the Active Directory User Account entitlement is later removed from the user's DirXML-EntitlementRef attribute, the Revoke User Account policy is processed.
4. The policies trigger the granting or revoking of access to the entitled resource. In the diagram above, the Grant User Account policy triggers the creation of a user account in Active Directory.

1.1.2. Why Use Entitlements

Both role-based provisioning and workflow-based provisioning require the use of entitlements. If you use any of these User Application provisioning methods, you must use entitlements.

If you are not using the User Application for role-based or workflow-based provisioning, you might still want to use Role-Based Entitlements (RBEs) through the Entitlements Service driver. Using Role-Based Entitlements enables you to remove the business logic or decision-making from your driver policies. In the example used in [How Entitlements Work](#), the Active Directory driver policies include only the information required to grant or revoke an Active Directory user account. The decision about whether or not a user receives an Active Directory user account is handled through the entitlement agent, not the driver policies. In this case, the entitlement agent is the Entitlements Service driver.

Removing the business logic from drivers provides several benefits:

- If you have multiple drivers that are the same (for example, multiple Active Directory drivers) and your business logic changes, you don't need to change the logic in each driver. The logic only needs to change in the entitlement agent.
- You can use any of the three entitlement agents to grant an entitlement to a user. You can even use all three entitlement agents together. However, you should have only one entitlement agent handle an entitlement for a given user. For example, you could have an Active Directory User Account entitlement granted to a user by the Entitlement driver and a Linux User Account entitlement granted to the same user through the User Application's Role Service driver. However, you should not have the same entitlement (for example, the Active Directory User Account) managed by both the Entitlement driver and the User Application's Role Service driver. Doing so can cause unintended granting and revoking of the entitlement.

1.1.3. Drivers with Preconfigured Entitlements

The following drivers include configuration files that already contain entitlements and the policies required to implement them. These entitlements support the most common scenarios, including granting and revoking user accounts, groups, and e-mail distribution lists.

- Active Directory: Grant and revoke accounts, group membership, Exchange Mailbox
- OpenText eDirectory: Grant and revoke accounts and group membership.
- GroupWise: Grant and revoke accounts, grant and revoke members of distribution lists
- LDAP: Grant and revoke user accounts and group memberships
- Linux and UNIX: Grant and revoke accounts
- Lotus Notes: Grant and revoke user accounts and group memberships
- RACF: Grant and revoke accounts
- SharePoint: Grant and revoke accounts and group membership
- SAP User Management and Portal: Grant and revoke accounts, group membership, and OOTB

These are example entitlements and policies. You can use them if they meet your needs. If not, you can modify them to meet your needs or use them as examples as you implement additional entitlements.

1.1.4. Entitlements Formats

The entitlements format has changed for version 4.0 and later. Before 4.0, the entitlement format was single-valued. For 4.0 and later, the entitlement value is multivalued.

The following information explains when each type of entitlement format is supported:

Previous to 4.0 Environment: There are two different formats in the earlier 4.0 environments:

- **Not Using Role Mapping Administrator (previous to 3.6.1):** These versions only support the legacy entitlement format. All granting agents assume the entitlement contains a single value. For a list of element agents, see [Why Use Entitlements](#).
- **Using Role Mapping Administrator:** Driver supporting the Role Mapping Administrator can define a legacy format for structured parameters in the format of: `<param>param1=value1|param2=value2|paramN=valueN</param>`

The granting agents are instructed to use the structured format of the EntitlementConfiguration Resource object. The SAP User Management Fan-Out driver is the only driver that supported this format before 4.0.

Identity Manager 4.0 and later Environment: Two different options support the formats in 4.0 and later:

- **Using Role-Based Entitlements:** Only the legacy format is supported with Role-Based entitlements (RBE). All entitlements are granted through RBE must be set to legacy. Some reports are not available when using the legacy format. Reports joining and the granted entitlements against the actual state in the managed system do not work using the legacy format.
- **Not Using Role-Based Entitlements:** The legacy and 4.0 and later formats are supported. Full reporting capabilities are available when the 4.0 and later format is used.

Mixed Previous to 4.0 and Identity Manager 4.0 and later Environments: If you have a mixed environment during migrations, the legacy format is the only supported entitlement format. All entitlements granted through RBE or previous to 4.0 agents must be set to legacy. Some reports are not available when using the legacy format. Reports joining and the granted entitlements against the actual state in the managed system do not work using the legacy format.

1.1.5. Understanding Identity Manager 4.0 and Later Entitlement Formats

For 4.0 and later, the entitlement value is multivalued. The format of storing the entitlement parameter has changed. The new entitlement parameters are formatted as a JSON string as follows:

```
{"ID": "aspencer-cell"}
```

JSON string allows the use of name=value pairs and introduces the concept of well-known fields where the same type of information can be stored using the same identifier and can be retrieved easily by a consuming application (for example, Role-Based Provisioning Module, Role Mapping Administrator, Identity Reporting Module). JSON also addresses the issue of escaping special characters in values. It is a well-known format, and many JSON parsers and generators are available for use.

Mandatory Fields for JSON

ID	Display Name	Sample Value	Mandatory	Description
ID	Primary ID	f1e84f2a796 4614eaa4540 7c724e3a98	Yes	The OpenText Identity Manager drivers use the primary ID to identify the corresponding entity in the connected system. For example, the Active Directory driver uses the objectGUID in Active Directory as the identifier. However, the GUID is not of any value to other applications.

ID	Display Name	Sample Value	Mandatory	Description
ID2	Secondary ID	CN=Domain Users,CN=Users,DC=yourcompany,DC=msft	No	The OpenText Identity Manager drivers can use the secondary ID to identify the corresponding entity in the connected system. This ID is not used by OpenText Identity Manager but might provide value to other consuming applications of the entitlement. Active Directory can identify an object by its objectGUID and DN. DN is a valuable secondary ID for Active Directory entitlements.

ID	Display Name	Sample Value	Mandatory	Description
LIID	Logical Instance ID	S7ICLNT800	No	The fan-out drivers can use the Logical Instance ID to know if the entitlement is granted to the logical instance of the application they connect to.

You can add more fields if entitlements require additional fields. For example, adding validity dates (valid from and valid to) or other. The names of the new fields must start with an underscore "_" as shown in [Optional Fields for JSON](#).

Optional Fields for JSON

ID	Display Name	Sample Value	Mandatory	Description
_FROM	Valid From	20100318	No	A sample custom field the SAP User Management driver uses for role entitlements.
_TO	Valid To	20100319	No	A sample custom field the SAP User Management driver uses for role entitlements.

Entitlement Parameters

The following are examples of entitlement parameters for drivers:

Active Directory driver:

```
{
  "ID": "f1e84f2a7964614eaa45407c724e3a98",
  "ID2": "CN=Domain Users,CN=Users,DC=yourcompany,DC=msft"
}
```

SAP User driver:

```
{
  "ID": "A_BEW_ALL",
  "LIID": "S7ICLNT800",
  "_FROM": "20100318",
  "_TO": "20100319"
}
```

1.2. Checklist for Implementing Entitlements

Use the following checklist to ensure you complete all the tasks required to implement entitlements for an OpenText Identity Manager driver. The tasks are listed in the recommended order of completion, but you can change the completion order if necessary.

Entitlements Checklist

Details

- Enable the driver to support entitlements

The driver must be configured to listen for entitlement events. You enable the driver by modifying the driver filter to add the DirXML-EntitlementRef attribute to the User class.

The following drivers are already enabled for entitlements. You do not need to complete this task for these drivers:

- Active Directory
- GroupWise
- LDAP
- Linux and UNIX
- Lotus Notes
- RACF

For enablement instructions, see [Enabling Entitlements on a Driver](#).

- Create entitlements

Entitlements represent resources in connected systems. While creating an entitlement, it is created on the driver associated with the connected system where the entitlement's resource is located.

For information, see [Creating Entitlements](#).

Details

- Create policies to support the entitlements

Entitlements are implemented by adding new driver policies or modifying existing policies.

For information, see [Creating Policies to Support Entitlements](#).

- Set up an entitlement agent to manage the entitlements

The entitlement agent is responsible for granting or revoking entitlements for users. You can use any of the following entitlement agents:

- **Entitlements Service Driver:** Manages entitlements based on events that occur in the Identity Vault. For information, see the [Entitlement Service Implementation Guide](#).
- **User Application Roles-Based Provisioning:** Manages entitlements based on roles assigned to users. For instructions, see the [Install and Upgrade Guide for Linux](#) or the [Install and Upgrade Guide for Windows](#).
- **User Application Workflow-Based Provisioning:** Manages entitlements through provisioning workflows.

1.3. Enabling Entitlements on a Driver

A driver must be enabled to use entitlements. You can enable a driver by modifying the driver filter to add the DirXML-EntitlementRef attribute to the User class.

The following drivers are enabled for entitlements and do not require manual enablement:

- Active Directory
- GroupWise
- LDAP
- Linux and Unix
- Lotus Notes
- RACF

1.3.1. Using Designer to Enable Entitlements

Designer is the recommended tool for creating entitlements (see [Creating Entitlements in OpenText Designer](#)). During the entitlement creation process, you are given the opportunity to have it automatically enable the driver to support entitlements. You should do so at that time.

1.4. Creating Entitlements

Because entitlements represent resources in a connected system, each entitlement must be created on the driver associated with the connected system. For example, to create an entitlement for an Active Directory User Account, you would create it on the Active Directory driver that connects to the directory where you want the account created.

The following sections provide instructions for creating entitlements in Designer. Although you can use either tool to create entitlements, we strongly recommend that you use it. It includes an Entitlement Wizard that creates the entitlement XML from the information you provide in the wizard.

- [Sample Entitlements for the Active Directory Driver](#)
- [Creating Entitlements in OpenText Designer](#)

1.4.1. Sample Entitlements for the Active Directory Driver

By default, the Active Directory driver includes the entitlements listed below. You can use these entitlements as examples of the types of entitlements you might want to create for other drivers.

- **User Account Entitlement:** Grants or revokes an account in Active Directory for the user. When the account is granted, the user is given an enabled logon account. When the account is revoked, the logon account is either disabled or deleted, depending on how the driver is configured.
- **Group Membership Entitlement:** Grants or revokes membership in a group in Active Directory. When this entitlement is granted, the user is added to the group in Active Directory. When this entitlement is revoked, the user is removed from the group in Active Directory.
- **Exchange Mailbox Entitlement:** Grants or revokes an Exchange mailbox for the user in Microsoft Exchange.

1.4.2. Creating Entitlements in OpenText Designer

OpenText Designer is the recommended tool for creating entitlements.

It provides an Entitlement Wizard that steps you through the creation of entitlements. The wizard creates the entitlement XML from the information you provide.

1. In the Modeler view for your OpenText Designer project, right-click the driver icon , then click **New > Entitlement** to launch the Entitlement Wizard.
2. Fill in the following fields:

Name: Specify the name for the entitlement. This is the name used for the entitlement object in the Identity Vault. The name is seen in both OpenText Designer and OpenText Identity Console.

Display Name: By default, the entitlement agents that consume the entitlements use the name specified in the **Name** field. If you want to specify a different name for the entitlement agent, deselect **Use this name for the display name**, and enter a name in the **Display Name** field.

For example, the GroupWise driver's default configuration file includes a predefined GroupWise account entitlement. The entitlement's name is *gwAccount*, and its display name is *GroupWise User Account*.

Description: Specify any information you want to use to describe the entitlement. This field is optional.

3. Click **Next**.

You can create two types of entitlements:

- *Valued entitlements* - contain values that are sent to the driver policy that enforces the entitlement.
 - *Valueless entitlements* - do not contain any values to pass.
4. Select **No** if the entitlement does not need to include values, then click **Finish**. If the Add to Filter dialog box is displayed, select **Yes**, then click **OK** to enable the entitlement for the driver. Skip the remaining steps in this section.

or

Select **Yes** if the entitlement needs to include values, click **Next**, then continue with the next step.

There are two types of values that you can use with valued entitlements:

- **Administrator-defined values:** Defined by you or another administrator. You can define a specific list of values from which the entitlement consumer must select, or you can designate a free-form value that the entitlement consumer defines.
- **Application Query values:** Supplied by the application to which the driver is connected. For example, the GroupWise driver's default configuration includes a predefined GroupWise Distribution List entitlement that enables users to be added to GroupWise distribution lists. The available distribution lists are discovered through a query of the GroupWise system.

5. Select **Administrator-defined values** if the valued entitlement requires values defined by you, click **Next**, then skip to [Administrator-Defined Entitlements with Value Lists](#) or [Administrator-Defined Entitlements without Value Lists](#).

or

Select **Values from an application query** if the valued entitlement requires values that must be discovered by querying the connected application, click **Next**, and then skip to [Valued Entitlement that Queries an External Application](#).

Administrator-Defined Entitlements with Value Lists

The example in the following procedure is an administrator-defined entitlement that allows you to select a listed entry. This type of entitlement is best used through Workflow entitlements rather than Role-Based Entitlements.

1. Make sure you've completed [Step 1](#) through [Step 5](#). The following steps start where those steps ended.
2. In the Define Values dialog box, make sure that **Yes** is selected.
3. In the Entitlement Value field, type the value you want to add to the list, then click **Add**. Repeat this step for each value you want to add to the list, then click **Next**.

New Entitlement Wizard

Define Values

Entitlement values can be taken from a defined list, or they can be entered by entitlement consumers.

Do you want to define a list of values?

☒ Yes

Choose this if you want to define a list of values for the entitlement consumers to choose from.

☐ No

Choose this if you want entitlement consumers to define their own values.

Entitlement Value:

Building D Add

Defined List:

Building A
Building B
Building C

Remove
Edit

? < Back Next > Finish Cancel

In this example, the values are corporate buildings: Building A through Building D. Through an entitlement client, such as an OpenText Identity Console Role-Based Entitlement task or the user application, users or defined-task managers can specify the building information, which is then included in an external application, such as OpenText eDirectory.

4. In the Assign Multiple Values dialog box, select **Yes** if you want the entitlements to be able to be granted to a user more than once and with different values, then click **Next**.

or

Select **No** if the entitlement can only be granted once, and click **Next**.

For example, you might only want to use an entitlement once to assign a building location to a user. However, because a user could belong to multiple groups, you

might require an entitlement that assigns a user to a group to be able to be used multiple times to assign the user to multiple groups.

5. You are asked if this entitlement is intended to be used by Role-Based Entitlement policies through OpenText Identity Console. If you want this entitlement to be granted or revoked automatically, select **Yes** to the Role-Based Entitlements question, click **Next**, and continue with [Step 6](#).

or

If you want the granting or revoking of this entitlement to be a manual process (approved by someone), select **No** to use the User Application, then skip to [Step 7](#).

6. (Conditional) If you selected **Yes** to the Role-Based Entitlements question, you are asked if you want to use the Role-Based Entitlements priority to resolve any conflicts that might happen when this entitlement is assigned by different Role-Based Entitlement Policies with different values. You can resolve the conflict by using the Role-Based Entitlements priority or by merging the values.

Merging the values merges the entitlements of all involved Role-Based Entitlement policies, so if one policy revokes an entitlement but another policy grants an entitlement, the entitlement is eventually granted. Solving conflicts by priority works if you need to ensure that only one policy is applied to this entitlement at any time.

7. Click **Finish**.

8. If you see the Add To Filter dialog box, answer **Yes** if you want the driver to listen for this entitlement. This enables entitlements for the driver. The DirXML-EntitlementRef attribute allows the driver filter to listen for entitlement activities, which is necessary to use the entitlements you are creating.

or

If you don't want to see the Add To Filter window on entitlements you create for any driver in OpenText Designer, select **Remember Selection - Don't Prompt Again**, and click **OK**. However, after the attribute is added to this driver filter, you won't see the Add To Filter window again.

Before you can edit this entitlement, you are asked to save the editor's changes before continuing. After the editor is saved, the entitlement displays in the Modeler view.

Administrator-Defined Entitlements without Value Lists

The example in the following procedure is an administrator-defined entitlement that forces the administrator to type a value. You can use this kind of entitlement if you do not have all of the information at the initial setup, so you cannot create a value list.

1. Make sure you've completed [Step 1](#) through [Step 5](#). The following steps start where those steps ended.
2. In the Define Values dialog box, select **No** to the question "Do you want to define a list of values?" on the Define Values page, then click **Next**.

Selecting this option allows the administrator or users to type in a value. Using this option can be risky however since wrong or misspelled information can cause the value to be incorrect and the action in the entitlement to fail.

3. In the Assign Multiple Values dialog box, select **Yes** if you want the entitlements to be able to be granted to a user more than once and with different values, then click **Next**.

or

Select **No** if the entitlement can only be granted once, then click **Next**.

For example, you might only want to use an entitlement once to assign a building location to a user. However, because a user could belong to multiple groups, you might want an entitlement that assigns a user to a group to be able to be used multiple times to assign the user to multiple groups.

4. You are asked if this entitlement is intended to be used by Role-Based Entitlement policies through OpenText Identity Console. If you want this entitlement to be granted or revoked automatically, select **Yes** to the Role-Based Entitlements question, click **Next**, then continue with [Step 5](#).

or

If you want the granting or revoking of this entitlement to be a manual process (approved by someone), select **No** to use the User Application, then skip to [Step 6](#).

5. (Conditional) If you select **Yes** to the Role-Based Entitlements question, you are asked if you want to use the Role-Based Entitlements priority to resolve any conflicts that might happen when this entitlement is assigned by different Role-Based Entitlement Policies with different values. You can resolve the conflict by

either using the Role-Based Entitlements priority, or by merging the values. This example uses priority.

Merging the values merges the entitlements of all involved Role-Based Entitlement policies, so if one policy revokes an entitlement but another policy grants an entitlement, the entitlement is eventually granted. Solving conflicts by priority works if you need to ensure that only one policy is applied to this entitlement at any time.

6. Click **Finish**.

7. If you see the Add To Filter window, answer **Yes** if you want the driver to listen for this entitlement. This enables entitlements for the driver. The DirXML-EntitlementRef attribute allows the driver filter to listen for entitlement activities, which is necessary in order to use the entitlements you are creating.

or

If you don't want to see the Add To Filter window on entitlements you are creating for any driver in OpenText Designer, select **Remember Selection - Don't Prompt Again**, then click **OK**. However, after the attribute is added to this driver filter, you won't see the Add To Filter window again.

Before you can edit this entitlement, you are asked to save the editor's changes before continuing. After the editor is saved, the entitlement displays in the Modeler view.

Valued Entitlement that Queries an External Application

1. Make sure you've completed [Step 1](#) through [Step 5](#). The following steps start where those steps ended.
2. On the Define Application Query page, fill in the fields to define the query and map the query results.

New Entitlement Wizard

Define Application Query

Define an application query that will return value(s) to be used by this entitlement.

Enter a class to query

Class:

Enter a base DN and select a scope to search from

Base DN: Scope:

Map query results to values used by entitlement consumers

Value From Query	Value Used by Entitlement Consumers
Source Distinguished	"Display-Name" shown to entitlement consumers
Description	"Description" shown to entitlement consumers
Association	"Value" used by policy

< Back Next > Finish Cancel

Enter a class to query: Click the **Schema Browser** button on the right side of the Class entry. The Schema Browser shows you the Classes in the OpenText eDirectory namespace that are available. If you know the name of the Class type you want to query, click to select a selection in the **Classes** tab, then start typing the Class name. The browser jumps to the alphabetical order of what you type. Select the Class name, then click **OK**.

Enter a base DN and select a scope to search from: Type the distinguished name (DN) of the directory base where you want to start the search. Select the scope (subtree, entry, or subordinates).

Map query results to the values used by entitlement consumers: Map the query results from the connected system to values that entitlement consumers can use.

- **Display Name:** Defines the attribute that displays in the list of values. Click the drop-down button on the **Display Name shown to entitlement consumers** list to see a list of Attributes associated with the class you selected through the Schema Browser. The list includes both the Attributes and the Inherited Attributes for the selected class.

- **Description:** Defines the attribute that displays as a description for that value. For the description, select **Description** from the Value drop-down list to map the query results from the connected system to the entitlement.
- **Value:** Defines the attribute or token that is the actual value. The **Value** entry is not seen in the entitlement consumer, but it is the value that is assigned when the entitlement is granted or revoked. In this case, choose **Association**.

If you do not use the **Schema Browser** button when selecting the class, you see only two selections in the **Value From Query** lists: **Association** and **Source Distinguished Name**. If these attributes suit your needs, use them. You can also type the attribute name into the text field. However, if you want to select the attributes from the lists, use the **Schema Browser** button when selecting a class for the query. You see the attributes and inherited attributes for the selected class.

3. When you've finished defining the query, select **Next**.
4. In the Assign Multiple Values dialog box, select **Yes** if you want the entitlements to be able to be granted to a user more than once and with different values, then click **Next**.

or

Select **No** if the entitlement can only be granted once, then click **Next**.

For example, you might only want to use an entitlement once to assign a building location to a user. However, because a user could belong to multiple groups, you might require an entitlement that assigns a user to a group to be able to be used multiple times to assign the user to multiple groups.

5. You are asked if this entitlement is intended to be used by Role-Based Entitlement policies. If you want this entitlement to be granted or revoked automatically, select **Yes** to the Role-Based Entitlements question, click **Next**, then continue with [Step 6](#).

or

If you want the granting or revoking of this entitlement to be a manual process (approved by someone), select **No** to use the User Application, then skip to [Step 7](#).

6. (Conditional) If you select **Yes** to the Role-Based Entitlements question, you are asked if you want to use the Role-Based Entitlements priority to resolve any

conflicts that might happen when this entitlement is assigned more than once with different values. You can resolve it by using the Role-Based Entitlements priority or merging the values.

Merging the values merges the entitlements of all involved Role-Based Entitlement policies, so if one policy revokes an entitlement but another policy grants an entitlement, the entitlement is eventually granted. Solving conflicts by priority works if you need to ensure that only one policy is applied to this entitlement at any time. This example uses priority.

7. Click **Finish**.

In the example shown in [Step 2](#), the query values look for the Source Distinguished Name attribute of the Class name of Group, starting from the Base DN (Blanston) and checking through the subtree from that beginning point. The values that come back from the query are similar to the following:

```
<instance class-name="Group" src-dn="o=Blanston,cn=group1">
  <association>o=Blanston,cn=group1</association>
  <attr attr-name="Description"> the description for
group1</attr>
</instance>
<instance class-name="Group" src-dn="o=Blanston,cn=group2">
  <association>o=Blanston,cn=group2</association>
  <attr attr-name="Description"> the description for
group2</attr>
</instance>
<instance class-name="Group" src-dn="o=Blanston,cn=group3">
  <association>o=Blanston, cn=group3</association>
  <attr attr-name="Description"> the description for
group3</attr>
</instance>
<!-- ... ->
```

The information received from the query fills in the various fields. For instance, the < display-name > field receives o=Blanston,cn=group1. The < description > field receives the description for Group1, and the < ent-value > field receives o=Blanston,cn=group1. Because more than one group exists and meets the query criteria, this information is also collected and shown as other instances of the query.

The association format value is unique for every external system, so the format and syntax are different for each external system queried.

8. If you see the Add To Filter window, answer **Yes** if you want the driver to listen for this entitlement. This enables entitlements for the driver. The DirXML-EntitlementRef attribute allows the driver filter to listen for entitlement activities necessary to use the entitlements you create.

or

If you don't want to see the Add To Filter window on entitlements you create for any driver in OpenText Designer, select **Remember Selection - Don't Prompt Again**, then click **OK**. However, you won't see the Add To Filter window again after the attribute is added to this driver filter.

1.5. Creating Policies to Support Entitlements

An entitlement gives a user permission to access the resource represented by the entitlement. However, to grant or revoke entitlements, you must create appropriate policies.

For information about creating policies, see the [Using Designer to Create Policies](#) and [Identity Manager Policies in Identity Console Guide](#).

By default, the Active Directory driver includes several entitlements and the policies required to support the entitlements. These policies are listed below. You can use these policies as examples of policy types you might need to create to support entitlements on other drivers.

- **Input Transform (driver level):** The Check Target Of Add Association For Group Membership Entitlements rule in this policy checks the target of “add-association” for group membership entitlements. Group membership entitlements assigned to users being created in Active Directory cannot be processed until the user is successfully created. Add-association signals that an object has been created by the driver in Active Directory. If the object is also tagged for group entitlement processing, it performs the work now.
- **Event Transform (Publisher channel):** The Disallow User Account Delete rule in this policy disallows deleting a user account in the Identity Vault. When you use the User Account Entitlement, managed user accounts are controlled by the entitlement in the Identity Vault. A delete in Active Directory does not delete the controlling object in the Identity Vault. A future change to the object in the Identity Vault or a merge operation might re-create the account in Active Directory.
- **Command (Subscriber channel):** The Command policy contains the following rules of entitlements:
 - The User Account Entitlement Change (Delete Option) rule. The User Account Entitlement grants the user an enabled account in Active Directory. Revoking the entitlement disables or deletes the Active Directory account, depending on the value you select for the **When account entitlement revoked** global variable. This rule executes when the entitlement is changing and you have selected the **Delete** option.

- The User Account Entitlement Change (Disable Option) rule. The User Account Entitlement grants the user an enabled account in Active Directory. Revoking the entitlement disables or deletes the Active Directory account, depending on the value you select for the **When account entitlement revoked** global variable. This rule executes when the entitlement is changing and you have selected the **Disable** option.
- The Check User Modify for Group Membership Being Granted or Revoked rule.
- The Check User Modify for Exchange Mailbox Being Granted or Revoked rule.
- **Matching (Subscriber channel):** This is the Account Entitlement: Do Not Match Existing Accounts rule for this policy. When you use the User Account entitlement with the OpenText Identity Manager user application or Role-Based Entitlements, accounts are created and deleted (or disabled) by granting or revoking the entitlement. The default policy does not match an existing account in Active Directory if the user is not entitled to an account in Active Directory. Modify or remove this rule if you want the entitlement policy to apply to matching accounts in Active Directory. This might result in the Active Directory account being deleted or disabled.
- **Creation (Subscriber channel):** The Creation policy contains the following rules of entitlements:
 - Account Entitlement: Block Account Creation When Entitlement Not Granted. When you use the User Account entitlement with the OpenText Identity Manager User Application or Role-Based Entitlements, accounts are created only for users who are specifically granted the account entitlement. This rule vetoes user account creation when the entitlement is not granted.
 - Identity Vault Accounts Are Enabled if the Login Disabled Does Not Exist.
 - Prepare To Check Group Entitlements After Add. Group entitlements are processed after the add completes, because the added object must exist to be added to a group. The add is flagged with an operational property that is checked in the input transform when the add processing completes.
 - Signal the Need To Check Exchange Entitlements After the Add.
 - Map User Name to Windows Logon Name. When userPrincipalName is configured to follow the OpenText eDirectory user name, set

userPrincipalName to its object name plus the name of the Active Directory domain.

1.6. Editing Entitlements in Designer

After you have created entitlements, you might need to edit them. The Entitlement Editor provides a graphical interface for editing entitlements. You can also edit the XML source directly.

Using the Entitlement Editor

1. From the Outline view, right-click an entitlement that appears under the Subscriber channel of the selected driver, then click **Edit**.

or

Double-click the entitlement icon to bring up the entitlement in the Entitlement Editor.

You can also right-click the driver icon in the Modeler view, then select **Edit Entitlements**.

2. If you have more than one entitlement for the selected driver, you see the Edit Entitlements window listing the available entitlements. Select an entitlement, then click **OK**. The entitlement appears in the Entitlement Editor.

Entitlement Editor

Application Query.id-provider.Driver Set:Blanston, Inc.

Name And Description

Specify a name and a description for the entitlement.

Name: Application Query

Display-Name: Application Query

Description: Looks for the Class name of Groupx

Multi-Value

If this entitlement is used to assign values, do you want this entitlement to be assignable more than once?

☒ Allow the entitlement to be assigned multiple times?

Role-Based Entitlements

If this entitlement will be consumed by Role-Based Entitlements, how do you want to resolve conflicts?

☒ The Role-based entitlement with priority will set the value

☐ Merge the values assigned

Values

Do you want...

☐ No values

☐ Administrator-defined values.

☒ Values from the application

Class: Group

Base DN: Blanston Scope: subtree

Value from Query	Entitlement Consumer Value
Source Distinguished Name	Display-Name
Description	Description
Association	Value

Entitlement Editor XML Source XML Tree

The Entitlement Editor view shows all pages and choices as of in the Entitlement Wizard, but the information is on one page.

- **Entitlement Editor:** Displays the full DN name for the entitlement. If there is a conflict with the entitlement name or some other error, a red icon appears to the left of the Entitlement Editor name, followed by an error message.
- **Name and Description:** Allows you to edit the name, display name, and description you specified to this entitlement.
- **Multi-Value:** Allows you to select if you want an entitlement to assign multiple times.
- **Role-Based Entitlements:** Allows you to select conflict resolution for Role-Based Entitlements. If you do not select Role-Based Entitlements, the **Role-based entitlements with priority** button is the default.
- **Values:** Allows you to define how values are defined: no values, administrator values, or values from an application.

The information that appears in the Entitlement Editor depends on what you initially defined in the entitlement. If you choose to edit a valueless entitlement, the **Values** heading displays **No Values**. If you are editing a valued entitlement and want to add values to a list, type the value in the **Value** field and click **Add**. If you want to remove a value, select the value in the **Values** list and click **Remove**.

If you don't want to select from a list, select **Administrator Defined Values** under the **Values** heading and leave the **Values** list blank. In the user application, you can fill in the value there.

3. When you have made your changes to the entitlement, click the **Save** icon in the upper left corner of OpenText Designer, or click the **X** on the entitlement's tab to display a Save Resource window, allowing you to save changes (**Yes/No/Cancel**).

Using the XML Source and XML Tree Views

1. From the Outline view, right-click an entitlement that appears under the Subscriber channel of the selected driver, then click **Edit**.

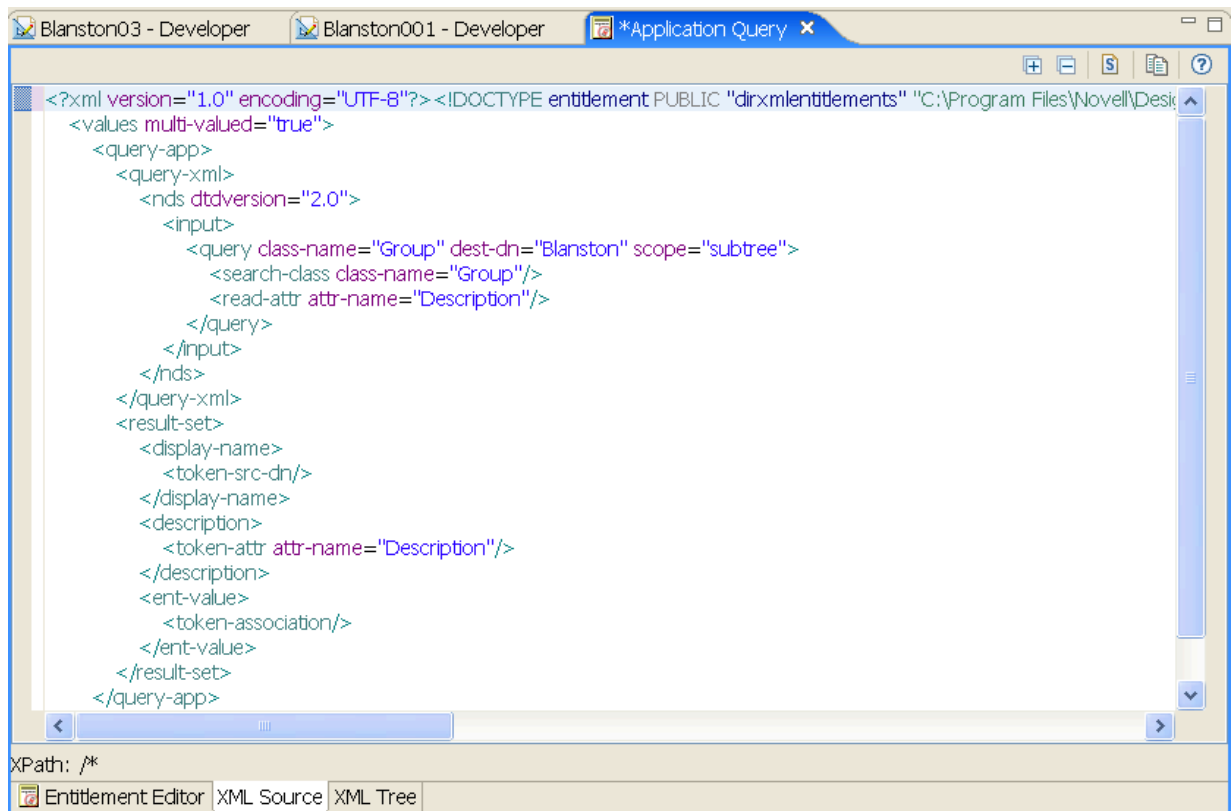
or

Double-click the entitlement icon to bring up the entitlement in the Entitlement Editor.

You can also right-click the driver icon in the Modeler view, then select **Edit Entitlements**.

- To see the XML Source view, click **XML Source** at the bottom of the Entitlement Editor view.

The XML Source view shows the XML code in a formatted state.



The upper right corner of the XML Source view has the following selections:

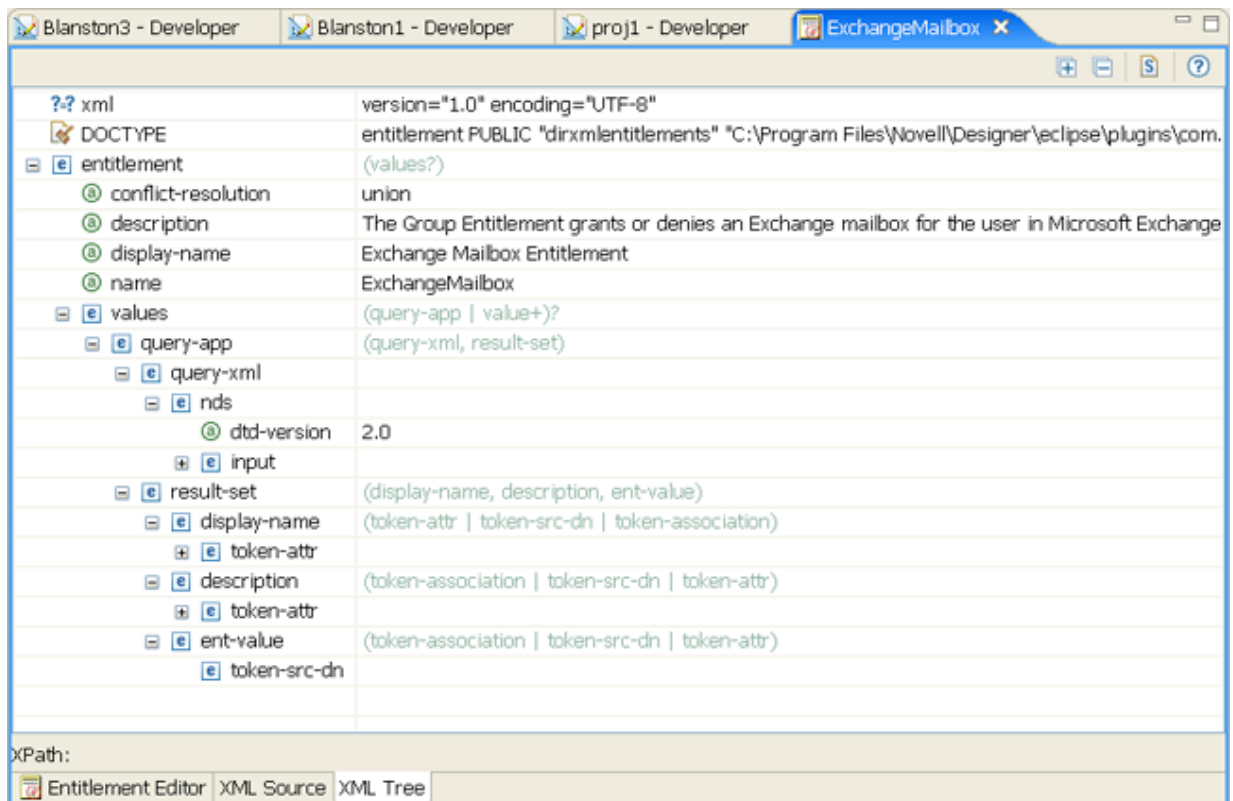
Name	Description
Expand All	Allows you to see all items under the item that you have selected.
Collapse All	Allows you to collapse all items that you have selected.
Attach XML Catalog Entry, XML Schema, or DTD	Allows you to attach an XML Catalog entry, an XML schema file, or a DTD (Document Type Definition) file. For default Windows installation, the DTD for entitlements is found under C:\Program Files\Novell\Designer\eclipse\plugins\com.novell.designer.idm.entitlements_1.1.0\DTD\dirxmlentitlements.dtd .
Copy XML to Clipboard	Allows you to copy highlighted XML code to the clipboard. This action removes the DOCTYPE element.
Find/Replace (Ctrl+F)	Ctrl+F brings up the Find/Replace window, which allows you to query text, structure, and XPath searches in a forward or backward direction. Other options include case sensitive, wrap search, whole word, incremental, and regular expressions search capabilities.
case-sensitive	Opens the Help view to the right of the XML Source view.

Right-clicking in the XML Source view brings up the following options:

- Undo Text Change (Ctrl+Z)
- Revert File
- Save
- Cut (Ctrl+X)
- Copy (Ctrl+C)
- Paste (Ctrl+V)
- Format the document or active elements
- Clear Validation Errors
- Validate
- Preferences

3. To see the XML Tree view, click **XML Tree** at the bottom of the Entitlement Editor view.

The XML Tree view is a tree control view of the XML source code. You can perform the same edits in this view as you can in the Entitlement Editor view or the XML Source view. To view the entitlement in XML Tree view, select **XML Tree** at the bottom of the Entitlement Editor view.



The upper right corner of the XML Tree view menu contains the following selections:

Name	Description
Expand All	Allows you to see all items under the item that you have selected.
Collapse All	Allows you to collapse all items that you have selected.
Attach XML Catalog Entry, XML Schema, or DTD	Allows you to attach an XML Catalog entry, an XML schema file, or a DTD (Document Type Definition) file. For default Windows installation, the DTD for entitlements is found under C:\Program Files\Novell\Designer\eclipse\plugins\com.novell.designer.idm.entitlements_1.1.0\DTD\dirxmlentitlements.dtd .
Find/Replace (Ctrl+F)	Brings up the Find/Replace window, which allows you to query text, structure, and XPath searches in a forward or a backward direction. Other options include case sensitive, wrap search, whole word, incremental, and regular expressions search capabilities.
Help	Opens the Help view to the right of the XML Tree view.

Right-clicking in the XML Tree view can bring up a number of different options. For example, right-clicking the highlighted value on the right side presents the following options:

- Undo
- Cut
- Copy
- Paste
- Delete
- Select All

Right-clicking an attribute on the left side in the XML Tree view presents the following options:

- Remove
- Edit the Selected Attribute
- Replace with a value

Depending on what you select on the left side in the XML Tree view, you see different options. For example, right-clicking an element presents the following options:

- Remove Element
- Add New Attribute
- Add to a Child Element a Comment, a Processing Instruction, a PCDATA, a CDATA Section, a new Element
- Add Before a Comment, a Processing Instruction, a PCDATA, a CDATA Section, a new Element
- Add After a Comment, a Processing Instruction, a PCDATA, a CDATA Section, a new Element

1.7. Writing Entitlements in XML

Entitlements, like other OpenText Identity Manager features and functionality, are implemented through XML code that is processed by its various components. OpenText Designer includes an Entitlements Wizard that creates the required XML code for an entitlement. It is recommended to use OpenText Designer to create entitlements.

To understand what needs to go into an entitlement, look at the preconfigured entitlements and policies in the Active Directory driver. You will need to examine the Entitlement DTD (Document Type Definition) and look at XML examples of writing entitlements based on the DTD.

The following sections provide information to help you create XML entitlement documents:

- [Entitlement Document Type Definition \(DTD\)](#)
- [Examples to Help You Write Your Own Entitlements](#)

1.7.1. Entitlement Document Type Definition

The following is the Entitlement Document Type Definition (DTD):

```
<!--
*****
*-->
<!-- DirXML Entitlements DTD <!-- Novell Inc. <!-- 1800 South
Novell Place
<!-- Provo, UT 84606-6194 <!-- Version=1.0.0 <!-- Copyright 2005
Novell, Inc. All rights reserved --> <!--
***** --
>
<!-- Entitlement definition stored in the XmlData attribute of a
DirXML-Entitlement object. -->
<!ELEMENT entitlement (values?)>
<!ATTLIST entitlement conflict-resolution (priority | union)
"priority" display-name CDATA #REQUIRED description CDATA
#REQUIRED >
<!ELEMENT values (query-app | value+)?>
<!ATTLIST values multi-valued (true | false) "true" >
<!ELEMENT value (#PCDATA)>
<!ELEMENT query-app (query-xml, result-set)>
<!ELEMENT query-xml ANY>
<!ELEMENT result-set (display-name, description, ent-value)>
<!ELEMENT display-name(token-attr | token-src-dn | token-
association)>
<!ELEMENT ent-value (token-association | token-src-dn | token-
attr)>
<!ELEMENT description (token-association | token-src-dn | token-
attr)>
<!ELEMENT token-association EMPTY> <!ELEMENT token-attr EMPTY>
<!ATTLIST token-attr attr-name CDATA #REQUIRED >
<!ELEMENT token-src-dn EMPTY>
<!-- Entitlement reference stored in the DirXML-EntitlementRef
attribute of a DirXML-EntitlementRecipient or a DirXML-
SharedProfile object. -->
<!ELEMENT ref (src?, id?, param?)>
<!ELEMENT param (#PCDATA)>
<!ELEMENT id (#PCDATA)>
<!ELEMENT src (#PCDATA)>
<!-- Entitlement result stored in the DirXML-EntitlementResult
attribute of a DirXML-EntitlementRecipient object. -->
<!ELEMENT result(dn, src, id?, param?, state, status,
```

```
msg?,timestamp)>
<!ELEMENT dn (#PCDATA)>
<!ELEMENT state (#PCDATA)>
<!ELEMENT status (#PCDATA)>
<!ELEMENT msg ANY> <!ELEMENT timestamp (#PCDATA)>
<!-- Cached query results stored in the DirXML-SPCachedQuery
attribute of a DirXML-Entitlement object. -->
<!ELEMENT items (item*)>
<!ELEMENT item (item-display-name?, item-description?, item-
value)>
<!ELEMENT item-display-name (#PCDATA)>
<!ELEMENT item-description (#PCDATA)>
<!ELEMENT item-value (#PCDATA)>
<!-- Representation of a DirXML-EntitlementRef within the DirXML
Script and within the operation-data of an operation in an XDS
document. -->
<!ELEMENT entitlement-impl (#PCDATA)>
<!-- ATTLIST entitlement-impl name CDATA #REQUIRED src CDATA
#REQUIRED id CDATA #IMPLIED state (0 | 1) #REQUIRED src-dn CDATA
#REQUIRED src-entry-id CDATA #IMPLIED -->
```

Explaining the Entitlement DTD

The Entitlement DTD is broken into five parts: definition, reference, result, cached query, and internal reference information. The heading is just a comment and is optional. In the DTD, the heading for the Entitlement Definition is:

```
<!-- Entitlement definition stored in the XmlData attribute of a
DirXML-Entitlement object. -->
```

Headings are followed by Elements (`ELEMENT`) and Attribute lists (`ATTLIST`). Below is a detailed explanation of the elements and attributes under the Entitlement Definition heading, which is the main heading you need to focus on when creating entitlements.

```
<!ELEMENT entitlement (values?)>
```

The root level element is `<entitlement>`, which can contain a single, optional, child `<values>` element. This is followed by the Attribute list, which includes `conflict-resolution`, `display-name`, and `description`. Conflict resolution uses Priority or Union attribute values.

conflict-resolution (priority | union) "priority"

Role-Based Entitlements use conflict resolution to determine what should happen when a valued entitlement is applied multiple times to the same object. For example, suppose that user U is a member of Entitlement Policy A and Entitlement Policy B, each of which reference the same valued entitlement E, but with a different set of values. Entitlement E of Entitlement Policy A has values (a, b, c). Entitlement E of Entitlement Policy B has a set of values (c, d, e).

The conflict resolution attribute decides which set of values should apply to user U. If the attribute is set to union, user U is assigned both sets of values (a, b, c, d, e). If it is set to priority, user U would get only one set of values, depending upon which Entitlement Policy has a higher priority.

If an entitlement is single-valued, conflicts must be resolved by priority, because a union of values results in more than one value being applied. Role-Based Entitlements presently uses this attribute; in the future, Workflow Entitlements might also use it.

display-name CDATA #REQUIRED description CDATA #REQUIRED

The literal entitlement name is not necessarily what an entitlement should display. The Display-name and Description attributes are intended for end-user display. (In OpenText Designer, you have an option to choose a display name for the entitlement instead of using the actual entitlement name.)

<!ELEMENT values (query-app | value+)?> <!ATTLIST values multi-valued (true | false) "true"

The `<values>` element is optional and indicates that an entitlement is valued. If you do not use this element, it means an entitlement is valueless. An example of a valued entitlement is an entitlement that grants a distribution list. An example of a valueless entitlement is an entitlement that grants an account in an application, such as the User Account entitlement that comes with the Active Directory driver.

Valued entitlements receive their values from three sources. One source is the external application (designated by the `<query-app>` element). Another is from a predefined list of enumerated values (one or more `<value>` elements). The third source is from the entitlement client (a `<values>` element with no `<value>` children). The examples are helpful in explaining the way values work.

Valued entitlements may be single-valued or multi-valued, and the default is multi-valued. It is the responsibility of the entitlement client to enforce this restriction.

<!ELEMENT value (#PCDATA)>

Entitlement values are untyped strings.

```
<!ELEMENT query-app (query-xml, result-set)>
```

If values come from an external application (such as an e-mail distribution list), you must specify an application query through the `<query-xml>` element. You extract the results from the query through the `<result-set>` element. We show two examples of this in [Example 2: Application Query Entitlement: External Query](#).

```
<!ELEMENT query-xml ANY>
```

XML queries are XDS-formatted. The `<query-xml>` command is used to find and read objects from the connected application. The functionality for DirXML rules, object migration, etc. depends on the driver's implementation of the query command. For more information about XML queries, see the [OpenText Identity Manager developer documentation on queries](#).

```
<!ELEMENT result-set (display-name, description, ent-value)> <!ELEMENT display-name(token-attr | token-src-dn | token-association)> <!ELEMENT ent-value (token-association | token-src-dn | token-attr)> <!ELEMENT description (token-association | token-src-dn | token-attr)> <!ELEMENT token-association EMPTY> <!ELEMENT token-attr EMPTY> <!ATTLIST token-attr attr-name CDATA #REQUIRED>
```

Use the `result set` element to help you interpret the result of an external application query. There are three pieces of data that are of interest: the display name of the value (the `display-name` child element), the value's description (the `description` child element), and the literal entitlement value (the `ent-value` child element), which is not displayed.

The token elements `<token-src-dn>`, `<token-association>`, `<token-attr>` are actually placeholders for XPATH expressions that extract the src-dn attribute value, association value, or any attribute values respectively from an XDS-formatted XML document. The DTD assumes that the query result is XDS.

Other Headings in the DTD

The remaining entitlement headings in the Entitlement DTD serve different functions, but they are not items that you need to focus on when creating an entitlement.

```
<!-- Entitlement reference stored in the DirXML-EntitlementRef attribute of a DirXML-EntitlementRecipient or a DirXML-SharedProfile object. →
```

The information stored in the Entitlement Reference portion of the DTD points to an entitlement object. This information is placed there by the managing agent (such as the Role-Based Entitlement driver, `Entitlement.xml`, or the Approval Flow driver,

UserApplication.xml). This is the triggering event for an action to take place in a connected system. You don't need to do anything with the DTD under this heading, but you can use this information to ensure that the entitlement object is being referenced.

<!-- Entitlement result stored in the DirXML-EntitlementResult attribute of a DirXML-EntitlementRecipient object. →

The Entitlement Result portion reports the results about whether an entitlement is granted or revoked. The information includes the state or status of the event and when the event is granted or revoked (through a time stamp). You don't need to do anything with the elements and attributes under this heading.

<!-- Cached query results stored in the DirXML-SPCachedQuery attribute of a DirXML-Entitlement object. →

The Entitlement Query portion contains the entitlement values that are gathered from an external application. This information can then be used again if the entitlement client needs to display this information. These values are stored in the DirXML-SPCachedQuery attribute of the Entitlement object. You don't need to do anything with the elements and attributes under this heading.

<!-- Representation of a DirXML-EntitlementRef within the DirXML Script and within the operation-data of an operation in an XDS document. →

Because the DTD defines values for more than one document, this EntitlementRef portion is actually not part of the Entitlement definition. You don't need to do anything with the elements and attributes under this heading.

1.7.2. Examples to Help You Write Your Own Entitlements

You can create valued and valueless entitlements. Valued entitlements can get their values from an external query, from an administrator-defined list, or free form. The following sections contain examples of four entitlements you can create.

Example 1: Account Entitlement: Valueless

```
<?xml version="1.0" encoding="UTF-8"?>
<entitlement conflict-resolution="priority"
  description="This is an Account Entitlement"
  display-name="Account Entitlement"/>
```

In this example, the valueless entitlement's name is Account. This is followed by the conflict-resolution line with the default setting of Priority, which in most cases means that if the entitlement is used by Role-Based Entitlements, the RBE with priority sets the value. (However, because this is an example of a valueless entitlement, valued settings don't apply.) The Entitlement description is **This is an Account Entitlement**, and the display name is Account Entitlement. This information is all you need to create an Account Entitlement, which you can then use to grant an account in an application.

The Active Directory driver with entitlements enabled has a UserAccount entitlement that Active Directory uses to grant or revoke a user account.

```
<?xml version="1.0" encoding="UTF-8"?>
<entitlement conflict-resolution="union"
  description="The User Account entitlement grants or denies an
  account in Active Directory for the user. When granted, the
  user
  is given an enabled logon account. When revoked, the logon
  account is either disabled or deleted depending on how the
  drive
  is configured."
  display-name="User Account Entitlement" name="UserAccount">
</entitlement>
```

In this example, the conflict resolution is Union, which allows the entitlement to merge the values that are assigned. (Again, valued settings don't apply to valueless

entitlements.) The **Description** field explains what this entitlement is used for and why it was created. This is useful information for those who perform future modifications to the entitlement. The actual name of the entitlement is UserAccount, while the `<display-name>` displays in a managing agent as User Account Entitlement.

Example 2: Application Query Entitlement: External Query

The Group and Exchange Mailbox entitlements that come with an entitlement-enabled Active Directory driver offer examples of application queries. Use this entitlement type when you need external information from a connected system to perform an event.

```
<?xml version="1.0" encoding="UTF-8"?>
<entitlement conflict-resolution="union"
  description="The Group Entitlement grants or denies membership
in
  a group in Active Directory. The group must be associated with
a
  group in the Identity Vault. When revoked, the user is removed
from
  the group. The group membership entitlement is not enforced on
the
  publisher channel: If a user is added to a controlled group in
Active Directory by some external tool, the user is not removed
by
  the driver. Further, if the entitlement is removed from the
user
  object instead of being simply revoked, the driver takes no
action."
  display-name="Group Membership Entitlement" name="Group">
  <values>
    <query-app>
    <query-xml>
    <nds dtd-version="2.0">
    <input>
    <query class-name="Group"
      scope="subtree">
    <search-class class-name="Group"/>
    <read-attr attr-name="Description"/>
    </query>
    </input>
    </nds>
    </query-xml>
    <result-set>
    <display-name>
    <token-src-dn/>
    </display-name>
    <description>
    <token-attr attr-name="Description"/>
    </description>
    <ent-value>
    <token-association/>
```

```
</ent-value>
</result-set>
</query-app>
</values>
</entitlement>
```

In this example, the Group entitlement uses Union to settle conflicts if the entitlement is applied more than once to the same object. The Union attribute merges the entitlements of all involved Role-Based Entitlement policies, so if one policy revokes an entitlement but another policy grants an entitlement, the entitlement is eventually granted.

The Group description is useful because of its detail, which explains what was set up through rules in the driver's policies. This description is a good example of the detail you need go into when defining entitlements in the first place.

The `<display-name>` is Group Membership Entitlement, which appears in the managing agents. The name is the Relative Distinguished Name (RDN) of the entitlement. If you don't define a display name, the entitlement's name is its RDN.

The initial query values look for the class name of Group at the top of the tree and continues through its subtrees. These values come from the connected Active Directory server and the application query starts at the `<nds>` tag. Under the `<query-xml>` tag, this query receives information similar to the following:

```
<instance class-name="Group" src-dn="o=Blanston,cn=group1">
  <association>o=Blanston,cn=group1</association>
  <attr attr-name="Description"> the description for
group1</attr>
</instance>
<instance class-name="Group" src-dn="o=Blanston,cn=group2">
  <association>o=Blanston,cn=group2</association>
  <attr attr-name="Description"> the description for
group2</attr>
</instance>
<instance class-name="Group" src-dn="o=Blanston,cn=group3">
  <association>o=Blanston, cn=group3</association>
  <attr attr-name="Description"> the description for
group3</attr>
</instance>
<!-- ... ->
```

Then, under the `<result-set>` tag, the information received from the query fills in the various fields. For instance, the `<display-name>` field receives `o=Blanston,cn=group1`. The `<description>` field receives the description for `group1`, and the `<ent-value>` field receives `o=Blanston,cn=group1`. Because more than one group existed and met the query criteria, this information was also collected and shown as other instances.



Note

The association format value is unique for every external system, so the format and syntax are different for each external system queried.

Another example is the Exchange Mailbox entitlement:

```
<?xml version="1.0" encoding="UTF-8"?>
<entitlement conflict-resolution="union"
  description="The Exchange Mailbox Entitlement grants or denies
an
  Exchange mailbox for the user in Microsoft Exchange."
  display-name="Exchange Mailbox Entitlement"
name="ExchangeMailbox">
  <values>
    <query-app>
      <query-xml>
        <nds dtd-version="2.0">
          <input>
            <query class-name="msExchPrivateMDB"
              dest-dn="CN=Configuration," scope="subtree">
              <search-class class-name="msExchPrivateMDB"/>
              <read-attr attr-name="Description"/>
              <read-attr attr-name="CN"/>
            </query>
          </input>
        </nds>
      </query-xml>
    <result-set>
      <display-name>
        <token-attr attr-name="CN"/>
      </display-name>
      <description>
        <token-attr attr-name="Description"/>
      </description>
      <ent-value>
        <token-src-dn/>
      </ent-value>
    </result-set>
  </query-app>
</values>
</entitlement>
```

In this example, the Exchange Mailbox entitlement uses Union to settle conflicts if the entitlement is applied more than once to the same object. The Union attribute merges the entitlements of all involved Role-Based Entitlement policies, so if one policy

revokes an entitlement but another policy grants an entitlement, the entitlement is eventually granted.

The description states that the entitlement grants or revokes an Exchange mailbox for the user in Microsoft Exchange, which is enough detail for what the entitlement does. The display name is Exchange Mailbox Entitlement, which appears in the managing agents. The name is the Relative Distinguished Name (RDN) of the entitlement. If you don't define a display name, the entitlement's name is its RDN.

The initial query values look for the class name of msExchPrivateMDB, which is a Microsoft Exchange function call that begins looking in container Configuration and continues through its subtrees. These values come from the connected Active Directory database and the application query starts at the `<nds>` tag. The class msExchPrivateMDB has no equivalent in OpenText eDirectory, so you need to be conversant in Microsoft Exchange function calls to make such a query. But the query is completed because of the rules and policies that are found in the Active Directory driver.

Entitlement consumers use the information that is retrieved by the query. For example, the entitlement value (`ent-value`) is passed to OpenText Identity Manager policies through the DirXML-EntitlementRef attribute. The display name and description information are displayed by the User Application and are stored in the DirXML-SPCachedQuery attribute.

Example 3: Administrator-Defined Entitlement: With Lists

The third example is an admin-defined entitlement that creates a grant or revoke event after you select a list entry.

```
<?xml version="1.0" encoding="UTF-8"?>
<entitlement conflict-resolution="union"
  description="This will show Administrator-defined Values">
  <display-name="Admin-defined Entitlement"/>
  <values multi-valued="true">
    <value>Building A</value>
    <value>Building B</value>
    <value>Building C</value>
    <value>Building D</value>
    <value>Building E</value>
    <value>Building F</value>
  </values>
</entitlement>
```

In this example, the entitlement name is `Admin-defined`, with a defined display name of `Admin-defined Entitlement`. (You only need to put in a display name if you want the display name to be different from the entitlement's RDN.) The conflict resolution line shows the setting of Union, which allows the entitlement to merge the values that are assigned.

The Entitlement description is `This will show Administrator-defined Values`. The multi-valued attribute is set to true, which allows the entitlement to assign a value more than once. In this example, the values are corporate building letters: Building A through Building F. Then, through an entitlement client or through the User Application, users or defined-task managers can specify the building information, which is then included in an external application, such as OpenText eDirectory.

Example 4: Administrator-Defined Entitlements: Without Lists

The fourth example is an admin-defined entitlement that forces the administrator to type a value before the entitlement can grant or revoke an event. You can use this kind of entitlement if you do not have all of the information to create a task list at the initial setup.

```
<?xml version="1.0" encoding="UTF-8"?>
<entitlement conflict-resolution="priority"
  description="There will be no pre-defined list">
  <values multi-valued="false"/>
</entitlement>
```

In this example, the entitlement name is **Admin-defined** (no list), and it uses the entitlement name as the displayed name because there is no display name entry. The conflict resolution is again set to the default of Priority, which means that if the entitlement is used by Role-Based Entitlements, the RBE with priority sets the value. Through an entitlement client, such as through the User Application, you specify the building information, which is then included in an external application, such as OpenText eDirectory.



© Copyright 2026 Open Text

For more info, visit <https://docs.microfocus.com>
